
autoMIL: Auditable Source-Level Research for Controlled Model Comparison

Shuolin Yin*

Yeonwoo Seo

Jun Ma

Abstract

A model leaderboard measures not only a method but also the training recipes, debugging, and search effort invested in that method. Coding agents can scale source-level research, yet their ability to edit and execute repositories does not by itself make research effort attributable or comparable across model families. We introduce **autoMIL**, a research-operations system that stores parent-addressed source overlays, reconstructs them in isolated git worktrees, accounts for failure-inclusive launched attempts, and separates validation-guided selection from held-out certification. We use this system to define **autobench**, a controlled audit of whether pathology multiple-instance-learning rankings are stable when each published method receives the same declared opportunity for autonomous source-level research inside a method-identity contract. The admissible search includes executable train-only recipe programs while freezing each method’s defining mechanisms; cross-method ranking is the primary estimand and within-method lift is explanatory. The current implementation passes 51 selected contract tests, and an audited static study record covers the declared 165-cell roster, but corrected multi-seed search and sealed cross-arm certification are still outstanding; this preprint therefore reports the framework, protocol, and present evidence boundary rather than a premature ranking result.

1 Introduction

Leaderboards compress a research process into a number. That number depends on the model, but also on optimizer choice, learning-rate schedules, stopping rules, data order, debugging, and the amount of search allocated to the model. The importance of these choices is established: algorithm tunability differs substantially across methods [1], and optimizer rankings can change when hyperparameter-optimization opportunity is included in the comparison [2]. A nominally architecture-centered benchmark can therefore measure a mixture of model quality and unequal research effort.

Language-model agents make this confound larger. MLAGentBench established iterative file editing, code execution, and result inspection as an evaluation setting [3]; AIDE formulates ML engineering as search over candidate programs [4]; and recent autonomous-research systems perform tree-structured exploration under explicit execution budgets [5–7]. Agent benchmarks increasingly save trajectories and use hidden evaluation [8, 9]. These systems show that autonomous source-level iteration is possible. They do not, however, automatically define the intervention needed to compare several existing published methods: what constitutes a method-identity-preserving edit, what consumes the research budget, which failures count, which result may guide the agent, and how the selected descendant is certified.

autoMIL addresses this operations and evidence problem. A proposed experiment is not a mutable checkout or a prose claim; it is a node with an exact parent, a source overlay, a declared intervention

*Correspondence: leo.yin@mail.utoronto.ca. **[PLACEHOLDER P-AUTH-1: confirm final author list, order, and affiliations before public release.]**

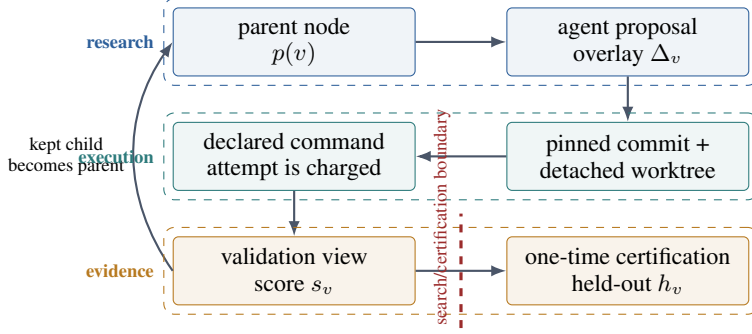


Figure 1: **The autoMIL lifecycle.** Each intervention is attributable to a parent and reconstructed from a source overlay. Search receives the validation-visible view; held-out fields are written to a separate certification location and excluded from descendant overlays. This is framework-mediated non-interference for normal orchestrated execution, not an operating-system security boundary against an adversarial same-user process.

surface, execution metadata, and a validation-visible result. The system reconstructs that node at a pinned commit, meters launched attempts even when they fail, and places held-out metrics on a separate certification path. Figure 1 shows the contract. The proposal policy is replaceable: autoMIL is the substrate that turns source-level interventions into attributable experimental evidence.

We make two contributions:

1. **An auditable research-operations framework.** autoMIL combines parent-addressed multi-file overlays, worktree reconstruction, declared file permissions, failure-inclusive attempt accounting, validation-only search, and separate held-out certification into one executable contract for repository-level autonomous research.
2. **A controlled pathology-MIL ranking audit.** autobench asks whether a native/no-search cross-model ranking agrees with the ranking after every method receives a matched declared search opportunity. Cross-arm ranking is essential; paired within-arm lift and anytime behavior explain any observed rank change or stability. The final certified outcome is not claimed in this version.

2 From Agentic Search to a Controlled Comparison

2.1 Research-opportunity estimand

Let $a \in \mathcal{A}$ denote a published method class and $c \in \mathcal{C}$ a study cell, such as a cohort, endpoint, encoder, and aggregator combination. An experiment node v records a parent $p(v)$, a source overlay Δ_v , an execution specification, and a validation score s_v . If selected for final evaluation, the node also has a separately revealed held-out result h_v . A per-cell cap B_c counts *launched attempts*; crashes, partial runs, and budget-killed jobs consume opportunity. Usable completed or partial results are counted separately.

For each method, the consumer predeclares an identity contract I_a over its defining inference operator, forward branches, and core training/loss mechanisms. The admissible set $\mathcal{V}_{a,c}^{\text{id}}$ preserves I_a while allowing both declared scalar configuration and executable train-only recipe logic, including sampling, adaptive schedules, optimizer/gradient policy, additive regularization, and stopping. Candidates that replace, delete, bypass, or zero a defining mechanism are outside this set and cannot be reported as results for method a .

For each cell, the native/no-search regime selects the predeclared native starting implementation and recipe. The matched-search regime selects an identity-preserving descendant using validation information subject to the same declared attempt cap:

$$v_{a,c}^0 \in \mathcal{V}_{a,c}^{\text{native}}, \quad (1)$$

$$v_{a,c}^*(B_c) = \arg \max_{v \in \mathcal{V}_{a,c}^{\text{id}} : N_{\text{launch}}(v) \leq B_c} s_v. \quad (2)$$

Table 1: **Closest-prior comparison.** “Repository” means multi-file source changes are the research object; “lineage artifact” means a selected candidate is explicitly reconstructed from its parent and delta; “final separation” means development selection is distinguished from held-out evaluation. The rightmost column is the target estimand, not a feature count.

System	Repository	Lineage artifact	Explicit budget	Final separation	Primary estimand
MLAgentBench [3]	yes	no	partial	partial	Agent performance on ML experimentation
AIDE [4]	partial	yes	partial	partial	Best candidate program
AIRA/AIRA2 [6, 7]	yes	yes	yes	yes	Research-agent search performance
MLRC-Bench [9]	yes	partial	yes	yes	Agent success on research challenges
AMID [10]	yes	partial	yes	yes	Autonomous medical-model development
PathBench-MIL / nnMIL [11, 12]	partial	no	partial	partial	Configured pathology-MIL pipelines
autoMIL + autobench	yes	yes	yes	yes	Cross-method rank response to matched source-level research

Certification produces two cross-method rankings,

$$\mathcal{R}_{\text{native}}(c) = \text{rank}_a(h_{v_{a,c}^0}), \quad \mathcal{R}_{\text{search}}(c; B_c) = \text{rank}_a(h_{v_{a,c}^*(B_c)}). \quad (3)$$

The primary analysis is their agreement, displacement, and uncertainty across cohorts. The paired lift $\Delta_{a,c} = h_{v_{a,c}^*} - h_{v_{a,c}^0}$ is secondary: it explains which methods moved and how much, but cannot replace the cross-arm ranking comparison.

2.2 What matched opportunity means

Matching launches defines a reproducible intervention, not universal fairness. It prevents a system from receiving extra replacements simply because its proposals crash, but it does not equalize GPU-hours, wall-clock time, model capacity, search difficulty, or the information content of one edit. We therefore use the narrower phrase *matched launched-attempt opportunity* and require a separate ledger of usable results, failure causes, elapsed time, accelerator use, and language-model cost. A time cap can be declared alongside the launch cap, but neither should be silently converted into “matched successful evaluations.”

The baseline also answers a specific question. Native recipes preserve how a method is normally entered into the comparison; a single shared optimizer would instead approximate an architecture-only ablation and can advantage methods whose published training procedure already resembles the shared recipe. Native recipes must therefore be sourced component by component, and benchmark-added choices must be labelled rather than presented as upstream defaults.

2.3 Closest prior work and the residual gap

Table 1 positions the contribution against the closest agentic systems and MIL infrastructure. Source-level editing, experiment trees, budgets, isolated execution, memory, and hidden evaluation are all prior art [3, 4, 6, 7, 9, 10]. Likewise, PathBench-MIL and nnMIL provide substantial pathology-MIL pipeline and benchmark infrastructure [11, 12]. We do not claim those ingredients individually. The residual contribution is their organization as a method-aware measurement instrument whose target is the sensitivity of a *cross-method ranking* to a controlled source-level research intervention.

3 autoMIL

3.1 Research layer: state before policy

The experiment record is a directed tree rather than a flat table of terminal scores. A node points to its parent and stores only changed files, description, status, resource metadata, and validation-visible metrics. A child is proposed against an explicit parent, so a result can be attributed to an exact research step and reconstructed independently of the agent’s conversation history. Persistent trajectories can inform later proposals, but the search policy is not fixed by the framework: a coding agent, heuristic proposer, or human can operate the same state machine.

Keep/discard decisions are made on a predeclared validation composite and margin. Held-out results are not an axis of tree expansion. This separation is important because repeated validation-guided selection can already overfit; allowing test feedback into the same loop would make the final comparison uninterpretable.

3.2 Execution layer: reconstruct, charge, collect

For a queued node, the runner creates a detached git worktree at the recorded base commit. It validates overlay targets, rejects absolute paths, traversal, and symlinks, optionally verifies SHA-256 digests, then copies only the declared delta. The consumer supplies the training command and result contract; the orchestrator supplies process isolation, GPU masking, logging, and result collection.

An attempt is charged when it is launched, before its scientific outcome is known. This ordering is deliberate. If charging depended on a usable metric, a brittle method implementation could obtain more proposals than a reliable one. Completed and partial results are nevertheless tracked separately, because matching launch opportunity without reporting yield would hide an important property of the research process. Orphan recovery is performed by the active daemon rather than read-only status commands, avoiding accidental mutation of live state.

3.3 Evidence layer: declared permissions and born-sealed results

The consumer declares editable and protected file surfaces. Submission hard-refuses a change that matches a protected path. This contract is file-based and intentionally keeps the generic framework free of pathology-specific architecture knowledge. The consumer owns the scientific contract: for C2 it must protect each method’s defining inference, forward, and core-loss mechanisms, expose a guarded train-only recipe seam, and classify every candidate as configuration-only, programmatic-recipe, identity-breaking, or invalid. Only the first two classes may enter the cross-method leaderboard. The current framework enforces file declarations, but the per-arm semantic invariants and runnable recipe seam remain implementation requirements.

Training writes a complete result to the node’s certification directory before placing a stripped, validation-visible result in the worktree. Descendant reconstruction excludes the certification subtree. At the end of the study, an explicit `certify` operation reveals the selected nodes’ held-out metrics once and warns that they must not be used for reselection. This design mediates the normal framework path; it is not a sandbox against arbitrary code running under the same operating-system account.

3.4 Consumer boundary and portability

autoMIL does not create a task-specific training program. A consumer supplies four items: a runnable command, a result conforming to the validation/held-out schema, a cell identity and budget, and editable/protected path declarations. The framework repository contains no pathology-specific decision rule; the `autobench` package is a separate consumer. Current evidence for this boundary is structural plus a synthetic-consumer round trip. It supports consumer-agnostic mechanisms, not the stronger claim that arbitrary projects require zero adaptation or that every execution backend has identical production semantics.

4 autobench: A Pathology-MIL Ranking Audit

4.1 Cohorts, tasks, and model regimes

Whole-slide histology is a useful stress test for research-effort sensitivity: slides contain many image patches, supervision is typically available only at the patient or slide level, cohorts are modest, and training conventions differ across MIL methods. `autobench` uses five cohorts from TCGA and CPTAC [13–15]. Each cohort contributes one classification endpoint and overall-survival prediction. Table 2 records the current audited planning counts; final sample sizes and attrition must be regenerated from frozen manifests before certification.

The tile-level grid crosses three pathology foundation-model encoders—UNI v2, Virchow2, and H-Optimus-1 [16–18]—with four published aggregator methods: CLAM [19], nnMIL SimpleMIL [12], attention MIL [20], and DTfD-MIL [21]. TITAN [22] is reported as a separate slide-level

Table 2: **Planned/audited cohort roster.** Counts are study-planning records, not a newly released cohort. HNSC has 431 cases, of which 414 are currently gradeable. OS denotes overall survival.

Cohort	Source	Classification endpoint	Cases	OS deaths
LUAD	TCGA	KRAS mutation (binary)	465	167
LGG	TCGA	IDH1 mutation (binary)	491	115
GBM	CPTAC	TP53 mutation (binary)	99	72
PDAC	CPTAC	immune subtype (3-class)	105	81
HNSC	TCGA	tumour grade (3-class)	431 (414)	205

regime because it jointly defines the representation and aggregation route; it is not another cell in the tile-encoder by aggregator factorial design.

With five cohorts, the declared roster contains 65 classification experiments and 100 survival experiments. The survival count is larger because SimpleMIL, ABMIL, and TITAN are evaluated with two predeclared survival objectives where supported. This 165-cell roster is a controlled sample of pathology MIL, not an exhaustive field leaderboard: prominent alternatives such as DSMIL and TransMIL are outside the current four-aggregator set [23, 24].

PLACEHOLDER P-GBM-1. Confirm whether CPTAC-GBM survival remains in the certified roster, is retained only as a labelled low-signal sensitivity case, or is removed before the roster is frozen. The 165-cell count and all denominators must be regenerated if this decision changes.

4.2 Native/no-search and matched-search regimes

Figure 2 shows the comparison. For the native/no-search arm, each framework begins from its upstream architecture and sourceable native recipe. Every component is labelled as upstream-declared, conditionally derived from an upstream rule, or benchmark-added. For the matched-search arm, a coding agent may propose source and recipe edits only within the declared method-identity contract and under the same launched-attempt opportunity in each comparison cell. This is not a scalar-only menu: admissible programmatic-recipe candidates may implement train-only sampling, adaptive schedules, optimizer or gradient policy, additive regularization, and stopping. They may not replace inference operators, introduce a new learnable forward branch, or remove a defining core training mechanism. Correctness fixes update the common protocol and trigger affected baseline regeneration; they are not counted as agent lift.

PLACEHOLDER P-BASE-1. Confirm the headline baseline policy. The present prose assumes per-method upstream/native recipes as the primary no-search ranking; if a standardized shared-recipe ranking is primary or co-primary, revise Sections 2.2 and 4.2 and preregister how unsupported framework-recipe combinations are handled.

This admissibility rule is settled before outcomes are observed, but its current implementation is incomplete. File-level permissions alone do not prevent a flag or loss weight from deleting a defining mechanism, and the declared structured policy path cannot yet execute the required train-only recipe programs. Identity-breaking candidates may be archived as evolved descendants, but they are excluded from C2 and never attributed to the published method.

PLACEHOLDER P-METHOD-1. Implement and test the settled method-identity contract before launching search: symmetric protected core files, machine-readable per-arm invariants, a consumer-side programmatic RecipePolicy hook called by protected trainers, and persistent four-way candidate classification.

4.3 Metrics, selection, and inference

Classification is reported with one-vs-rest macro AUC as the primary metric and balanced accuracy as a complementary class-sensitive metric. Survival is reported separately with concordance; classification and survival are never pooled into one performance number. Five-fold splits are patient-stratified,

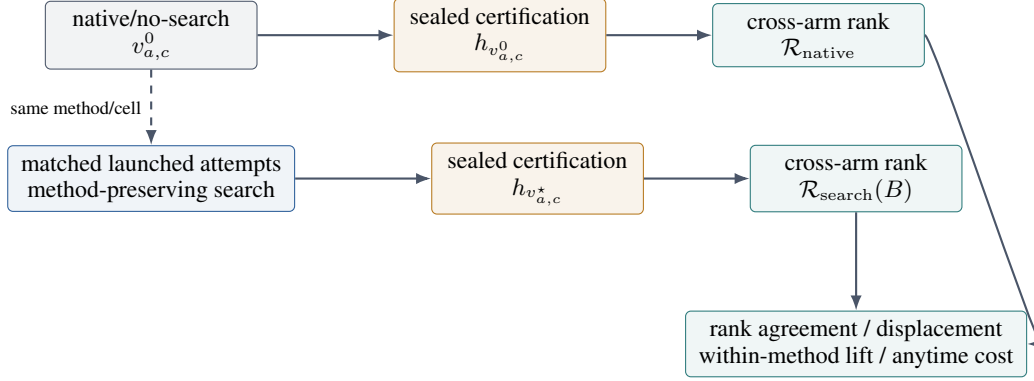


Figure 2: **Controlled ranking audit.** Native and searched descendants are certified on the same held-out split. The two cross-arm rankings are the primary objects. Paired lift, failure yield, and anytime cost explain the ranking response but do not replace it. The schematic is result-neutral.

and all competing arms within a cohort use the same patient partitions. Validation composites control search and promotion; held-out predictions and metrics enter only through certification.

The inferential unit cannot be the nominal set of 165 cells. Cells from the same cohort share patients and folds, and several cells also share representations. The final analysis will therefore report cohort-level rankings and paired model comparisons, then aggregate only with a declared dependence-aware procedure. Patient-level paired bootstrap intervals can quantify metric and rank uncertainty within a certified seed; repeated initialization seeds are needed to characterize training variability. Prior pathology-MIL evidence reports test-AUC swings of 10–15 points from initialization, batch order, and learning rate, and cautions that validation AUC may not predict test AUC reliably [25]. Because the current campaign record is single-seed, this version makes no statistical significance or seed-robust ranking claim.

PLACEHOLDER P-SEED-1. Confirm whether the preprint remains a descriptive single-seed audit or adds a repeated-seed confirmatory set. Until confirmed, retain no significance, stability, or seed-robustness claim.

4.4 Evidence and resource ledger

A certifiable comparison must release more than a leaderboard. For each method and cell, the evidence ledger includes: upstream commit and native recipe sources; benchmark-added components; frozen splits and feature identifiers; every parent-linked overlay; validation trajectories; selected nodes; launched and usable attempt counts; failures; wall-clock, GPU, and language-model costs; candidate admissibility class; and the one-time held-out result. This ledger makes an informative null possible. If rankings remain stable, the record shows the research intervention under which stability was observed; if they move, it locates the edits, costs, and methods responsible.

5 Validation and Current Evidence

We separate locally verified framework behavior from cluster-audit records and from outcomes that do not yet exist. This distinction is central to the preprint: protocol maturity is evidence, but it is not a substitute for the certified C2 result.

5.1 Does the framework enforce its contract?

A focused regression run executed 51 tests covering the worktree runner, born-sealed result path, launched-attempt cap, evaluation accounting, protected files, framework purity, and a synthetic-consumer round trip. All 51 passed in 4.80 seconds on the current checkout (Table 3). These tests directly exercise the named C1 mechanisms. They do not prove operating-system security, production equivalence among execution backends, or scientific generality.

Table 3: **Current framework validation.** This is a selected invariant suite, not a claim that the full workspace dependency matrix is green.

Invariant family	Result	Scope of evidence
Worktree reconstruction and safe overlay application	pass	Pinned checkout, digest/path handling, collection
Born-sealed validation/certification path	pass	Split result write and descendant exclusion
Launch-cap and usable-result accounting	pass	Failure-inclusive attempts, separate usable counts
Consumer-declared protected files	pass	Submission refusal for declared paths
Framework purity and synthetic consumer	pass	No pathology mechanism in core; one toy consumer
Selected total	51/51	4.80 s, current checkout

5.2 Is the static study substrate usable?

The external cluster audit reports 195 stored experiments, comprising every cell in the canonical 165-cell roster plus 30 off-roster LUAD experiments. Across 8,440 per-fold records it reports no fold-count anomaly and no non-finite value in the primary AUC, balanced-accuracy, or concordance fields. The underlying cluster artifacts are not present in this local checkout, so these are audited execution records rather than locally reproduced results.

The same audit exposed a consequential provenance defect: 102 of 195 serialized configuration files describe a generic recipe rather than the recipe that actually executed. Runtime provenance reconstruction is therefore the required source for historical recipes, and corrected native baselines must be regenerated before a leaderboard is published. We deliberately omit a rerun count because current planning documents disagree; the canonical roster and runtime provenance must be joined mechanically.

A separate implementation blocker remains in the CLAM survival path: it hard-codes AdamW while exposing the optimizer as a declared searchable field. An agent can therefore request a different optimizer and have the request silently ignored. This defect must be repaired and regression-tested before the matched campaign; none of the current text treats the affected runs as search-fidelity evidence.

5.3 Are the no-search baselines obvious strawmen?

A validation-only triage screen was available for 174 gradeable roster experiments. Relative to task-specific chance thresholds, 151 were more than 0.05 above chance, 19 were within 0.05, and four were at or below chance. All four subchance runs and 13 of the 20 near-chance cases were concentrated in GBM survival. This finding is useful but narrow: it argues against a globally broken study substrate and flags GBM survival as a predefined low-signal case. It does not establish that recipes are optimal, that the native ranking is unbiased, or that any model is superior. No held-out value was used for this screen.

5.4 What remains unestablished

Table 4 states the current evidence boundary. In particular, this preprint does not report a rank flip, rank stability, or a winning model. Those statements require corrected baselines, method-identity enforcement, a runnable programmatic-recipe channel, the matched campaign, repeated seeds, dependence-aware analysis, and one-time certification.

5.5 Certified cross-arm results

PLACEHOLDER P-RESULT-1. Insert the frozen native/no-search and matched-search cross-arm ranking table or figure only after canonical provenance reconciliation, selected-node freezing, and one-time held-out certification. Report classification and survival separately, with TITAN labelled as a different regime.

Table 4: **Evidence status for C2 as of 30 July 2026.** “Reported” denotes an external cluster-audit record whose raw artifacts are not in this checkout.

Object	Status	Evidence now	Required to unlock the claim
Canonical roster	reported	165/165 cells within 195 stored runs	Frozen manifest plus local artifact release
Fold/metric integrity	reported	8,440 records; no primary non-finite value	Re-run audit over released artifacts
Baseline viability	reported	validation-only 151/174 exceeded chance by > 0.05	Correct native recipes and held-out certification
Native cross-arm ranking	not yet established	historical configs misstate recipes	Canonical provenance join and corrected reruns
Matched-search ranking	not yet established	protocol and framework only	Method invariants, recipe channel, matched search, seed ledger, certification
Rank stability/change	not yet established	no certified paired rankings	Dependence-aware comparison of both rankings

PLACEHOLDER P-RESULT-2. Insert rank agreement/displacement and paired within-method lift with patient/cohort-aware uncertainty. Within-method lift must explain the cross-arm result; it must not replace the ranking comparison.

PLACEHOLDER P-RESULT-3. Insert anytime curves and the resource/failure ledger: launched attempts, usable outcomes, crash and budget-kill counts, wall-clock, GPU-hours, and language-model cost.

6 Discussion and Limitations

The novelty is the measurable question, not the ingredients. Worktrees, experiment graphs, resource caps, validation selection, and hidden tests all have clear precedents. autoMIL contributes their executable coordination around a different estimand: how an existing cross-method ranking responds when autonomous source-level research opportunity is declared, matched, and attributable. C2 is therefore not an optional demo. It is the scientific test that gives the integrated C1 contract meaning.

Internal validity. Five issues can invalidate the audit. First, historical configs do not faithfully encode every executed baseline recipe. Second, a permissive edit surface can transform one model family into another, while a scalar-only menu would fail to instantiate the intended source-level research opportunity. Third, single-seed training cannot distinguish a search effect from initialization variance. Fourth, validation-guided search can overfit even when test data remain sealed. Fifth, shared patients and folds make cell-level independence false. The corresponding remedies are, respectively, runtime-provenance reconstruction; per-method invariants plus a guarded programmatic train-only recipe seam; repeated seeds; one-time certification; and cohort/patient-aware paired analysis.

External validity. The benchmark covers five cohorts, three patch encoders, four tile-aggregator methods, and one separate slide-level model. It does not represent all pathology tasks or model families. The framework core contains no pathology-specific mechanism and passes one synthetic-consumer test, but this is not evidence for zero-effort adoption in arbitrary repositories. Local, SLURM, and Ray interfaces exist; real production-equivalence evidence across those backends remains outstanding.

Security and governance. The result firewall assumes normal execution through the orchestrator. Code with same-user filesystem access is outside the threat model. Moreover, an agent can exploit a poorly declared consumer surface without violating a framework rule. Protected inputs, splits, metric code, and method-defining components must therefore be reviewed as part of the study protocol. The framework records and enforces declarations; it cannot infer the scientific meaning of every source edit.

Outcome-neutral value. A stable ranking would show robustness to the declared research intervention, not that all recipe bias is absent. A changed ranking would quantify sensitivity under this budget, not establish a universally superior model. Either outcome is informative if accompanied by the intervention and resource ledger. Neither is informative if baselines, method identity, failures, or test exposure are repaired after inspecting the final result.

6.1 Conclusion

autoMIL is a framework for turning repository-level coding-agent interventions into reconstructable, parent-linked experimental evidence. Its current implementation mediates declared source permissions, failure-inclusive launch budgets, validation-only search signals, and a distinct held-out certification path. Focused invariant tests support those mechanisms, while their limits are explicit: the system is an auditable operations contract, not an operating-system sandbox or a proof of scientific generality.

autobench uses that contract to ask whether pathology-MIL cross-model rankings survive matched autonomous research opportunity. That cross-arm comparison is the second main contribution; within-arm lift explains it. The static roster and validation-only screen are sufficiently mature to expose baseline and inference problems, but the final answer awaits corrected native recipes, method-identity enforcement, the programmatic-recipe channel, repeated-seed evidence, and sealed certification. This preprint fixes the question and the evidence standard before revealing the outcome.

Broader Impact

Auditable experiment lineages can make autonomous ML research easier to inspect, reproduce, and challenge. They may also accelerate the production of seemingly authoritative comparisons. In computational pathology, premature leaderboards can redirect clinical-research effort even when the underlying cohorts are small or unrepresentative. We therefore keep patient-level data outside the release, separate validation from certification, disclose failed runs and compute, and avoid clinical-performance claims. autoMIL is intended for research operations; it does not validate a model for patient care.

References

- [1] Philipp Probst, Anne-Laure Boulesteix, and Bernd Bischl. Tunability: Importance of hyperparameters of machine learning algorithms. *Journal of Machine Learning Research*, 20(53):1–32, 2019. URL <https://www.jmlr.org/papers/v20/18-444.html>.
- [2] Robin M. Schmidt, Frank Schneider, and Philipp Hennig. Optimizer benchmarking needs to account for hyperparameter optimization. *Proceedings of the 37th International Conference on Machine Learning*, 119:8528–8537, 2020. URL <https://proceedings.mlr.press/v119/schmidt20a.html>.
- [3] Qian Huang, Jian Vora, Percy Liang, and Jure Leskovec. MAgentBench: Evaluating language agents on machine learning experimentation. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235, pages 20271–20309, 2024. URL <https://proceedings.mlr.press/v235/huang24y.html>.
- [4] Zhengyao Jiang, Dominik Schmidt, Dhruv Srikanth, Dixing Xu, Ian Kaplan, Deniss Jacenko, and Yuxiang Wu. AIDE: AI-driven exploration in the space of code. *arXiv preprint arXiv:2502.13138*, 2025. URL <https://arxiv.org/abs/2502.13138>.
- [5] Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist-v2: Workshop-level automated scientific discovery via agentic tree search. *arXiv preprint arXiv:2504.08066*, 2025. URL <https://arxiv.org/abs/2504.08066>.
- [6] Edan Toledo et al. AI research agents for machine learning: Search, exploration, and generalization in MLE-bench. In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/328b81881da145412f2bc56c998dfb6a-Abstract-Conference.html.
- [7] Karen Hambardzumyan et al. AIRA_2: Overcoming bottlenecks in AI research agents. *arXiv preprint arXiv:2603.26499*, 2026. URL <https://arxiv.org/abs/2603.26499>.
- [8] Hjalmar Wijk et al. RE-Bench: Evaluating frontier AI R&D capabilities of language model agents against human experts. *arXiv preprint arXiv:2411.15114*, 2024. URL <https://arxiv.org/abs/2411.15114>.

- [9] Yunxiang Zhang, Muhammad Khalifa, Shitanshu Bhushan, Grant Murphy, Lajanugen Logeswaran, Jaekyeom Kim, Moontae Lee, Honglak Lee, and Lu Wang. MLRC-Bench: Can language agents solve machine learning research challenges? In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/82c96f3c90741ef2c9b248e65d9b5db0-Abstract-Datasets_and_Benchmarks_Track.html.
- [10] Shengyuan Liu, Jia-Xuan Jiang, Boyun Zheng, Cheng Wang, Zipei Wang, Wentao Pan, Hongtao Wu, Houwen Peng, Yu Gu, Lichao Sun, and Yixuan Yuan. Towards autonomous and auditable medical imaging model development. *arXiv preprint arXiv:2607.10522*, 2026. URL <https://arxiv.org/abs/2607.10522>.
- [11] Siemen Brussee, Pieter A. Valkema, Jurre A. J. Weijer, Thom Doeleman, Anne M. R. Schrader, and Jesper Kers. Pathbench-mil: A comprehensive automl and benchmarking framework for multiple instance learning in histopathology. *arXiv preprint arXiv:2512.17517*, 2025. URL <https://arxiv.org/abs/2512.17517>.
- [12] Xiangde Luo, Jinxi Xiang, Yuanfeng Ji, and Ruijiang Li. nmmil: A generalizable multiple instance learning framework for computational pathology. *arXiv preprint arXiv:2511.14907*, 2025. URL <https://arxiv.org/abs/2511.14907>.
- [13] The Cancer Genome Atlas Research Network et al. The cancer genome atlas pan-cancer analysis project. *Nature Genetics*, 45:1113–1120, 2013. doi: 10.1038/ng.2764.
- [14] Lijun B. Wang et al. Proteogenomic and metabolomic characterization of human glioblastoma. *Cancer Cell*, 39(4):509–528.e20, 2021. doi: 10.1016/j.ccell.2021.01.006.
- [15] Lei Cao et al. Proteogenomic characterization of pancreatic ductal adenocarcinoma. *Cell*, 184(19):5031–5052.e26, 2021. doi: 10.1016/j.cell.2021.08.023.
- [16] Hanwen Xu et al. A whole-slide foundation model for digital pathology from real-world data. *Nature*, 630:181–188, 2024.
- [17] Eugene Vorontsov et al. A foundation model for clinical-grade computational pathology and rare cancers detection. *Nature Medicine*, 30:2924–2935, 2024.
- [18] Charlie Saillard et al. H-optimus-0. *arXiv preprint arXiv:2409.13031*, 2024.
- [19] Ming Y. Lu, Drew F. K. Williamson, Tiffany Y. Chen, Richard J. Chen, Matteo Barbieri, and Faisal Mahmood. Data-efficient and weakly supervised computational pathology on whole-slide images. *Nature Biomedical Engineering*, 5:555–570, 2021. doi: 10.1038/s41551-020-00682-w.
- [20] Maximilian Ilse, Jakub M. Tomczak, and Max Welling. Attention-based deep multiple instance learning. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80, pages 2127–2136, 2018. URL <https://proceedings.mlr.press/v80/ilse18a.html>.
- [21] Hongrun Zhang, Yanda Meng, Yitian Zhao, Yihong Qiao, Xiaoyun Yang, Sarah E. Coupland, and Yalin Zheng. DTFD-MIL: Double-tier feature distillation multiple instance learning for histopathology whole slide image classification. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 18802–18812, 2022. URL https://openaccess.thecvf.com/content/CVPR2022/html/Zhang-DTFD-MIL_Double-Tier_Feature_Distillation_Multiple_Instance_Learning_for_Histopathology_Whole_CVPR_2022_paper.html.
- [22] Tong Ding et al. Multimodal whole slide foundation model for pathology. *arXiv preprint arXiv:2411.19666*, 2024.
- [23] Bin Li, Yin Li, and Kevin W. Eliceiri. Dual-stream multiple instance learning network for whole slide image classification with self-supervised contrastive learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14318–14328, 2021.

- [24] Zhuchen Shao, Hao Bian, Yang Chen, Yifeng Wang, Jian Zhang, Xiangyang Ji, and Yongbing Zhang. Transmil: Transformer based correlated multiple instance learning for whole slide image classification. In *Advances in Neural Information Processing Systems*, volume 34, pages 2136–2147, 2021.
- [25] Ashar Mammadov et al. Reducing variability of multiple instance learning methods for digital pathology. In *Medical Image Computing and Computer Assisted Intervention – MICCAI 2025*, 2025. URL <https://papers.miccai.org/miccai-2025/0759-Paper4644.html>.

A Operational Contract

Result schema. A training process emits a status, validation-visible metric dictionary, validation-derived composite, elapsed time, peak memory, and an optional held-out block. The orchestrated writer places the full object under the certification directory and removes held-out fields from the workspace copy. The orchestrator, not the training script, writes the global result table.

Attempt semantics. The comparison cap increments on launch. Completed, partial, crashed, budget-killed, and cancelled outcomes remain distinguishable. A partial result can be scientifically usable but never refunds the launch that produced it. Per-cell reports must show at least launched attempts, usable outcomes, failure categories, wall time, accelerator-hours, and language-model spend.

Submission and reconstruction. Overlay paths must be relative to the project, remain within the project root, and not be symlinks. Consumer-declared protected paths are rejected at submission. Reconstruction starts from a pinned commit and can verify stored digests before applying the overlay. Certification files are excluded from descendant reconstruction.

B Reproduction Status and Commands

The focused validation reported in Table 3 was run from the repository root with:

```
.venv/bin/pytest tests/test_runner.py \
  tests/test_born_sealed_firewall.py \
  tests/test_launch_cap_enforcement.py \
  tests/test_daemon_eval_accounting.py \
  tests/test_submit_protected_files.py \
  tests/test_framework_purity.py \
  tests/test_synthetic_consumer_roundtrip.py -q
```

It produced 51 passed in 4.80s. We did not use the full workspace as a green validation gate: collection in the present local environment reported 856 collected tests and 114 collection errors, principally outside the focused contract command. The paper consequently makes only the selected-test claim.

The local checkout does not contain the cluster result tree underlying the 195-run audit. Releasing or mounting that tree and running a versioned audit reader are prerequisites for independently reproducing the reported static counts. The final campaign should additionally publish the exact environment, accelerator type, per-run timing, total compute, language-model/runtime versions, prompts or skills, and every selected source overlay.

PLACEHOLDER P-ARTIFACT-1. Add the permanent code/release URL, version or commit, environment lockfile, cluster artifact location, data/feature access instructions, asset licenses, and final compute/LLM-cost ledger.

C Planned Confirmatory Analysis

The following analysis must be frozen before final certification:

1. regenerate the canonical roster from dataset manifests and join every native result to runtime provenance;
2. repair the CLAM survival optimizer path so every declared search field controls the executed recipe, then add a regression test for silent drops;
3. implement and test per-method defining-mechanism invariants, symmetric protected cores, the consumer-side programmatic RecipePolicy seam, and four-way candidate classification;
4. execute matched launch caps with failures and resources retained;
5. repeat selected native and searched nodes across predeclared seeds;

6. certify native and searched selections once on identical held-out patient splits;
7. report cohort-specific cross-arm rankings, rank agreement and displacement, paired within-method lift, anytime curves, and patient/seed uncertainty without treating 165 cells as independent.

TITAN remains a labelled slide-level regime and is not silently pooled with the tile-encoder by aggregator factorial ranking.

D Author and Tooling Disclosure

The authors designed the framework, benchmark, contribution hierarchy, and evidence boundaries. Language models are an object of study because coding agents propose source-level experiments. Language-model assistance was also used to organize and edit this manuscript; all factual claims, citations, source anchors, and reported test commands remain the authors’ responsibility. No generated empirical value or mock result figure is used in the paper.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: The abstract, Introduction, and Table 4 separate established framework evidence from the unfinished ranking outcome.

Guidelines:

- The answer [\[N/A\]](#) means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [\[No\]](#) or [\[N/A\]](#) answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: Section 6 discusses baseline provenance, lineage drift, seeds, validation overfitting, dependence, external validity, and the security boundary.

Guidelines:

- The answer [\[N/A\]](#) means that the paper has no limitation while the answer [\[No\]](#) means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be

used reliably to provide closed captions for online lectures because it fails to handle technical jargon.

- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper introduces notation for the measurement protocol but contains no theorem or proof claim.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [No]

Justification: Appendix B reproduces the focused framework test, but the external cluster artifacts and final certified campaign are not yet released.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.

- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: The framework source is documented in the working repository, but a permanent public artifact and redistributable data/feature package are not yet cited in this version.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [No]

Justification: Section 4 and Appendix C specify the intended design, but corrected recipes, seed schedule, frozen lineage rules, and final resource settings remain incomplete.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.

- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: No inferential claim is made. Multi-seed and dependence-aware uncertainty are explicitly required before the final ranking claim.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [No]

Justification: The focused CPU test time is reported, but the cluster campaign's complete accelerator, wall-clock, failed-run, and LLM-cost ledger is not available locally.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The work is a research-operations and retrospective benchmark study; the paper avoids clinical-use claims and states its evidence boundaries.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: The Broader Impact section covers transparency benefits, premature-leaderboard risk, patient-data handling, and non-clinical scope.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: No high-risk model, scraped dataset, or generative media asset is released by this paper.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [No]

Justification: The papers and cohort sources are credited, but asset-by-asset versions and license terms must be added with the final public artifact.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [No]

Justification: `autoMIL` and `autobench` are new code assets, but the permanent release, version, license manifest, and complete campaign documentation are not yet provided in the paper.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The study uses no crowdsourcing and recruits no participants.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.

- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The paper reports no new intervention or participant recruitment; final secondary-use governance language must accompany the frozen cohort manifests.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: The core method uses coding agents, and Appendix B plus the Author and Tooling Disclosure describe their methodological and writing roles.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.